Files

..
sample_data
landsat_features_training.csv
landsat_features_validation.csv
submission_final.csv
submission_part2.csv
submission_template.csv
terraclimate_features_training.csv
terraclimate_features_validation....
water_quality_training_dataset.csv

Disk 86.94 GB available

[5] ✓ 3s

```
!pip install lightgbm scikit-learn pandas numpy
```

```
Requirement already satisfied: lightgbm in /usr/local/lib/python3.12/dist-packages (4.6.0)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from lightgbm) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2026.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
```

[6] ✓ 0s

```
import pandas as pd

# Load all datasets
landsat_train = pd.read_csv('/content/landsat_features_training.csv')
landsat_val = pd.read_csv('/content/landsat_features_validation.csv')

terra_train = pd.read_csv('/content/terraclimate_features_training.csv')
terra_val = pd.read_csv('/content/terraclimate_features_validation.csv')

water_df = pd.read_csv('/content/water_quality_training_dataset.csv')

submission = pd.read_csv('/content/submission_template.csv')

# Quick check
print("Landsat Train:", landsat_train.shape)
print("Landsat Validation:", landsat_val.shape)
print("Terra Train:", terra_train.shape)
print("Terra Validation:", terra_val.shape)
print("Water Dataset:", water_df.shape)
print("Submission:", submission.shape)
```

```
Landsat Train: (9319, 9)
Landsat Validation: (200, 9)
Terra Train: (9319, 4)
Terra Validation: (200, 4)
Water Dataset: (9319, 6)
Submission: (200, 6)
```

Files

..

sample_data

landsat_features_training.csv

landsat_features_validation.csv

submission_final.csv

submission_part2.csv

submission_template.csv

terraclimate_features_training.csv

terraclimate_features_validation....

water_quality_training_dataset.csv

Disk 86.94 GB available

[7] ✓ 6s

```python
#Import libraries
import pandas as pd
import numpy as np

from lightgbm import LGBMRegressor
from sklearn.model_selection import KFold
from sklearn.metrics import r2_score
```

[8] ✓ 0s

```python
#Load datasets
water = pd.read_csv("water_quality_training_dataset.csv")

landsat_train = pd.read_csv("landsat_features_training.csv")
landsat_test = pd.read_csv("landsat_features_validation.csv")

climate_train = pd.read_csv("terraclimate_features_training.csv")
climate_test = pd.read_csv("terraclimate_features_validation.csv")

submission = pd.read_csv("submission_template.csv")
```

[9] ✓ 0s

```python
#Check sizes:
print(water.shape)
print(landsat_train.shape)
print(climate_train.shape)
```

```
(9319, 6)
(9319, 9)
(9319, 4)
```

[10] ✓ 0s

```python
# Convert date columns in ALL datasets

water["Sample Date"] = pd.to_datetime(water["Sample Date"], dayfirst=True)

landsat_train["Sample Date"] = pd.to_datetime(
    landsat_train["Sample Date"], dayfirst=True
)

landsat_test["Sample Date"] = pd.to_datetime(
    landsat_test["Sample Date"], dayfirst=True
)

climate_train["Sample Date"] = pd.to_datetime(
```

Files

..

sample_data

landsat_features_training.csv

landsat_features_validation.csv

submission_final.csv

submission_part2.csv

submission_template.csv

terraclimate_features_training.csv

terraclimate_features_validation....

water_quality_training_dataset.csv

Disk 86.94 GB available

[10] ✓ 0s

```
climate_train["Sample Date"] = pd.to_datetime(
    climate_train["Sample Date"], dayfirst=True
)

climate_test["Sample Date"] = pd.to_datetime(
    climate_test["Sample Date"], dayfirst=True
)

submission["Sample Date"] = pd.to_datetime(
    submission["Sample Date"], dayfirst=True
)
```

[11] ✓ 0s

```
print(water["Sample Date"].dtype)
print(landsat_train["Sample Date"].dtype)
print(climate_train["Sample Date"].dtype)
```

```
datetime64[ns]
datetime64[ns]
datetime64[ns]
```

[12] ✓ 0s

```
#Merge Landsat features
train = water.merge(
    landsat_train,
    on=["Latitude","Longitude","Sample Date"],
    how="left"
)
```

[13] ✓ 0s

```
#Merge climate features
train = train.merge(
    climate_train,
    on=["Latitude","Longitude","Sample Date"],
    how="left"
)
```

[14] ✓ 0s

```
#Prepare submission dataset
test = submission.merge(
    landsat_test,
    on=["Latitude","Longitude","Sample Date"],
    how="left"
```

[14]
✓ 0s

```
)

test = test.merge(
    climate_test,
    on=["Latitude","Longitude","Sample Date"],
    how="left"
)
```

[15]
✓ 0s

```
#Feature engineering
for df in [train, test]:

    df["year"] = df["Sample Date"].dt.year
    df["month"] = df["Sample Date"].dt.month
    df["dayofyear"] = df["Sample Date"].dt.dayofyear

    df["lat_lon"] = df["Latitude"] * df["Longitude"]
    df["lat_sq"] = df["Latitude"] ** 2
    df["lon_sq"] = df["Longitude"] ** 2

    df["season"] = (df["month"] % 12) // 3 + 1
```

[16]
✓ 0s

```
#Define targets
targets = [
    "Total Alkalinity",
    "Electrical Conductance",
    "Dissolved Reactive Phosphorus"
]
```

+ Code + Text

[17]
✓ 0s

```
#Select feature columns
drop_cols = targets + ["Sample Date"]

features = [c for c in train.columns if c not in drop_cols]

features = [c for c in features if c in test.columns]
```

[18]
✓ 0s

```
#Check feature count:
print("Number of features:", len(features))
```

```
Number of features: 16
```

[19]
✓ 0s

```python
#Handle missing values
train = train.fillna(train.median(numeric_only=True))
test = test.fillna(train.median(numeric_only=True))
```

[20]
✓ 0s

```python
#Additional spatial features
for df in [train, test]:

    df["distance_origin"] = np.sqrt(
        df["Latitude"]**2 + df["Longitude"]**2
    )

    df["distance_center"] = np.sqrt(
        (df["Latitude"] + 30)**2 +
        (df["Longitude"] - 25)**2
    )

    df["lat_abs"] = np.abs(df["Latitude"])
    df["lon_abs"] = np.abs(df["Longitude"])
```

[21]
✓ 0s

```python
#Spatial clustering
from sklearn.cluster import KMeans

coords = train[["Latitude","Longitude"]]

kmeans = KMeans(n_clusters=20, random_state=42)

train["geo_cluster"] = kmeans.fit_predict(coords)

test["geo_cluster"] = kmeans.predict(test[["Latitude","Longitude"]])
```

[22]
✓ 0s

```python
# Recompute features after all feature engineering

features = [c for c in train.columns if c not in targets + ["Sample Date"]]

features = [c for c in features if c in test.columns]

print("Total features used:", len(features))
```

```
Total features used: 21
```

[23]
✓ 0s

```python
#Update feature list
drop_cols = targets + ["Sample Date"]

features = [c for c in train.columns if c not in drop_cols]
features = [c for c in features if c in test.columns]

print("Total features:", len(features))
```

```
Total features: 21
```

[24]
✓ 3s

```python
# Install XGBoost
!pip install xgboost
```

```
Requirement already satisfied: xgboost in /usr/local/lib/python3.12/dist-packages (3.2.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from xgboost) (2.0.2)
Requirement already satisfied: nvidia-nccl-cu12 in /usr/local/lib/python3.12/dist-packages (from xgboost) (2.29.7)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from xgboost) (1.16.3)
```

[25]
✓ 0s

```python
#Import XGBoost
from xgboost import XGBRegressor
```

[26]
✓ 1s

```python
#Feature importance filtering
temp_model = LGBMRegressor(
    n_estimators=500,
    learning_rate=0.05
)

temp_model.fit(train[features], train["Electrical Conductance"])

importances = pd.Series(
    temp_model.feature_importances_,
    index=features
)

top_features = importances.sort_values(
    ascending=False
).head(60).index.tolist()

print("Selected features:", len(top_features))
```

```
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.003082 seconds.
```

```
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.003082 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 3540
[LightGBM] [Info] Number of data points in the train set: 9319, number of used features: 21
[LightGBM] [Info] Start training from score 485.004146
Selected features: 21
```

```
#Update feature list
features = top_features
```

```
#Ensemble training function
def train_model(target):

    X = train[features]
    y = train[target]

    kf = KFold(n_splits=5, shuffle=True, random_state=42)

    oof = np.zeros(len(train))
    preds = np.zeros(len(test))

    for tr_idx, val_idx in kf.split(X):

        X_tr, X_val = X.iloc[tr_idx], X.iloc[val_idx]
        y_tr, y_val = y.iloc[tr_idx], y.iloc[val_idx]

        # LightGBM model
        lgb = LGBMRegressor(
            n_estimators=2500,
            learning_rate=0.02,
            max_depth=10,
            num_leaves=64,
            subsample=0.8,
            colsample_bytree=0.8,
            random_state=42
        )

        lgb.fit(X_tr, y_tr)

        lgb_pred = lgb.predict(X_val)
        lgb_test = lgb.predict(test[features])
```

```
            colsample_bytree=0.8,
            random_state=42=
        )

        lgb.fit(X_tr, y_tr)

        lgb_pred = lgb.predict(X_val)
        lgb_test = lgb.predict(test[features])

        # XGBoost model
        xgb = XGBRegressor(
            n_estimators=2000,
            learning_rate=0.03,
            max_depth=8,
            subsample=0.8,
            colsample_bytree=0.8,
            random_state=42
        )

        xgb.fit(X_tr, y_tr)

        xgb_pred = xgb.predict(X_val)
        xgb_test = xgb.predict(test[features])

        # Ensemble predictions
        val_pred = (lgb_pred + xgb_pred) / 2
        test_pred = (lgb_test + xgb_test) / 2

        oof[val_idx] = val_pred
        preds += test_pred / kf.n_splits

    score = r2_score(y, oof)

    print(target, "CV R2:", score)

    return preds, score
```

```
# Train all targets and print CV scores clearly

cv_scores = {}

# Ensure the train_model function (defined in cell G4Yd1Dx_7t_H) is executed before running this cell
```

```python
# Train all targets and print CV scores clearly

cv_scores = {}

# Ensure the train_model function (defined in cell G4Yd1Dx_Zt_H) is executed before running this cell.
# The error `ValueError: too many values to unpack (expected 2)` typically means
# the function is not returning the expected number of values.
# The `train_model` function should return two values: `preds` and `score`.

for target in targets:
    preds, score = train_model(target)
    test[target] = preds
    cv_scores[target] = score

print("\nCross Validation Scores")
print("----------------------")

for k, v in cv_scores.items():
    print(f"{k}: {v:.4f}")

print("\nAverage CV R2:", round(sum(cv_scores.values()) / len(cv_scores), 4))
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
Dissolved Reactive Phosphorus CV R2: 0.7206006392085331

Cross Validation Scores
----------------------
Total Alkalinity: 0.8632
Electrical Conductance: 0.8816
Dissolved Reactive Phosphorus: 0.7206

Average CV R2: 0.8218
```

```
#Post-processing
test["Dissolved Reactive Phosphorus"] = test[
"Dissolved Reactive Phosphorus"
].clip(lower=0)

test["Electrical Conductance"] = test[
"Electrical Conductance"
].clip(lower=0)

test["Total Alkalinity"] = test[
"Total Alkalinity"
].clip(lower=0)
```

```
#Remove negative chemistry values
test["Dissolved Reactive Phosphorus"] = test[
"Dissolved Reactive Phosphorus"
].clip(lower=0)

test["Electrical Conductance"] = test[
"Electrical Conductance"
].clip(lower=0)

test["Total Alkalinity"] = test[
"Total Alkalinity"
].clip(lower=0)
```

```
#Save submission
```

```
#Save submission
submission_output = test[
[
"Latitude",
"Longitude",
"Sample Date",
"Total Alkalinity",
"Electrical Conductance",
"Dissolved Reactive Phosphorus"
]
]

submission_output.to_csv("submission_part2.csv", index=False)
```

```
# Create final submission aligned with template

submission_final = submission.copy()

submission_final["Total Alkalinity"] = test["Total Alkalinity"].values
submission_final["Electrical Conductance"] = test["Electrical Conductance"].values
submission_final["Dissolved Reactive Phosphorus"] = test["Dissolved Reactive Phosphorus"].values

submission_final.to_csv("submission_final.csv", index=False)

print("Final submission saved as submission_final.csv")
```

```
Final submission saved as submission_final.csv
```

+ Code + Tex

```
print(submission_final.head())
print(len(submission_final))
```

```
    Latitude  Longitude Sample Date  Total Alkalinity  Electrical Conductance  \
0 -32.043333  27.822778  2014-09-01        132.650333              252.113728
1 -33.329167  26.077500  2015-09-16        248.253550              551.923853
2 -32.991639  27.640028  2015-05-07         83.267614              240.546815
3 -34.096389  24.439167  2012-02-07         25.497367              359.218826
4 -32.000556  28.581667  2014-10-01         97.644052              230.414146

   Dissolved Reactive Phosphorus
0                      25.438589
1                      42.162132
2                      37.165888
```

Files

..

sample_data

landsat_features_training.csv

landsat_features_validation.csv

submission_final.csv

submission_part2.csv

submission_template.csv

terraclimate_features_training.csv

terraclimate_features_validation....

water_quality_training_dataset.csv

```
   Dissolved Reactive Phosphorus
0                      25.438589
1                      42.162132
2                      37.165888
3                      10.805037
4                      19.343205
200
```

[35]

```
print(test.shape)
print(submission.shape)
```

```
(200, 25)
(200, 6)
```

NOTE:

Dataset not included due to confidentiality (EY Hackathon)

Refer to 'output_preview.pdf' for full execution results